Dynamic Programming & Reinforcement Learning: Assignment 2

Vrije Universiteit Amsterdam - Group 14 - 24/11/2023

Abdullah Öztürk
2682881

Ángel A. Gil
2788988

1 Introduction

In this assignment, we model a replacement problem for a system that deteriorates over time as a Markov Reward Chain (MRC). We then compute the stationary distribution & find the long-run average replacement costs through different methods. Finally, we try to find the optimal policy for a system where we are given the option for preventive replacement at a reduced cost.

2 The problem at hand. Defining the state space

Our system starts from *State 1*, with failure probability 0.1. For every step, there are two possibilities:

- **System fails**: If this happens, we go back to *State 1* from *State n*, with cost 1. $P(S(1)|S(n)) = 0.09 + 0.01n$.
- **System does not fail**: In this case, we advance to the next state with no cost. $P(S(n+1)|S(n)) = 0.91 - 0.01n$

The resulting state space of our MRC, shown in Figure 1, has 91 states: the number of time units until we reach guaranteed failure (provided that the system does not fail before, which is highly unlikely), including the initial state. The Markov property holds because $P(S(n+1)|S(n), S(n-1), ..., S(1)) = P(S(n+1)|S(n))$.

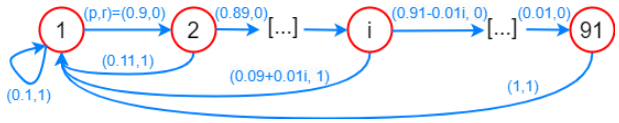

Figure 1: Initial formulation of the problem as a MRC.

3 Stationary distribution & long-run average replacement costs

In this section, we will attain the long-run average replacement costs ϕ^* by three different methods: 1) computing the stationary distribution by using $\pi^T = \pi^T P$, 2) simulating the system for a long period and 3) by solving the average-cost Poisson equation.

3.1 Computing the stationary distribution

Since we modeled the system as a MRC, we can use $\pi^T = \pi^T P$ to express state 1 to 91 as follows:

$$\begin{cases} \pi(1) = 0.9\pi(2) + 0.1\pi(1) \\ \pi(2) = 0.89\pi(3) + 0.11\pi(1) \\ \vdots \\ \pi(n) = (0.91 - 0.01n)\pi(n+1) + (0.09 + 0.01n)\pi(1) \\ \vdots \\ \pi(91) = \pi(1) \end{cases}$$

There are 91 different variables and infinite many solutions to this system (because the system is linearly dependent). However, there is also a normalization factor, which added to the system of equations $\phi_* = \sum_{i=1}^{91} \pi(i)r(i)$ leads to a unique solution. With this the long-run average replacement costs are approximately **0.1461**. Figure 2 shows the stationary distribution for the first 30 states, since the probability of reaching further states was computationally equal to 0.

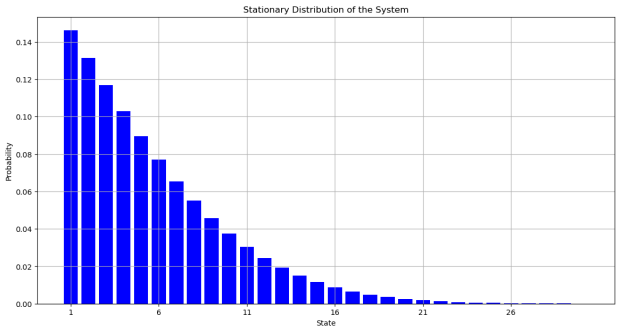

Figure 2: Initial formulation of the problem as a MRC.

3.2 Simulating the system

To substantiate the stationary distribution and the long-run average replacement costs, a high-level simulation was conducted. It abstracts the system's degradation over time, tracking the frequency of failures and the ensuing costs. The algorithm iterates through numerous cycles, with the system either progressing to a worse state or resetting to the initial state upon failure. Failures are simulated using a random number generator against the evolving failure probability. We set the number of iterations to 10^6 and ran the simulation process 10 times, obtaining a mean of simulated average replacement cost of 0.1461 with a standard deviation of 0.0002, which is quite robust and yields the same value as the obtained with the previous method. Finally, we also made one final test with 10^8 iterations, which took close to 10 minutes to run on CPU for the 10 simulations (opposed to $\approx$ 5 seconds for the previous testing). The results of this final test were an average of 0.1461 with std of $3 * 10^{-5}$, which further confirms the convergence to the same ϕ.

3.3 Solving the average-cost Poisson equation

The Poisson equation is as follows: $V_*(x)+\phi_* = r(x)+\sum_y p(y|x)V_*(y)$. With this we will get 91 equations:

$$\begin{cases} V(0) + \phi = r_0 + P_{10}(0)V(1) + P_{00}(0)V(0) \\ V(1) + \phi = r_1 + P_{21}(1)V(2) + P_{01}(1)V(0) \\ \vdots \\ V(90) + \phi = r_{90} + P_{90}(90)V(0) \end{cases}$$

We have got 92 variables (containing ϕ). If we suppose $V(0) = 0$, then the coefficient matrix is of dimensions $92X92$ and the reward a $92X1$ vector, so we get a unique solution. We use the linear algebra solver of *numpy* to find total expected reward starting from $x = 0$ and long-term average replacement costs ϕ. By using Poisson Equations, we obtain 0.1461 for the long-run average replacement costs one more time. This is expected, as the three methods should reach the same ϕ if correctly implemented.

4 The preventive replacement problem

In this new problem, we are given the option to make a preventive replacement with costs of 0.6. This means, that any state, we are given the option to go back to the first state at a reduced cost, compared to the cost that would be incurred if the system reaches failure by itself. Figure 3 shows the resulting model, for which we are going to find the optimal policy π^* through policy and value iteration. This way, we will know the best decision at each state in order to minimize the average expected cost.

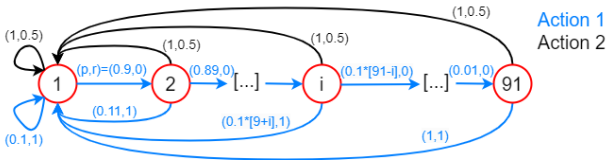

Figure 3: Updated system with replacement included.

4.1 Policy iteration

In our approach, the policy iteration method is utilized to determine the most cost-effective policy—whether to wait or replace—across all states in our system model. This iterative algorithm begins with an initial assumption where each state opts to wait, which corresponds to Action 1. Our objective is to minimize the sum of the immediate cost and the expected future costs, denoted by $V + \phi$ for each state's decision.

We construct a two-layer Poisson equation that represents the state transitions and associated costs for both waiting (*action* 1) and preemptive replacement (*action* 2) actions. Using the transition probabilities and rewards specific to our system, we calculate the policy's cost implications. Iteratively, we then assess and update the policy, selecting the action with the lower value of $V + \phi$ at each state—indicative of lower long-term costs.

The iteration persists until the policy remains unchanged, signaling the attainment of an optimal policy. In our implementation, it is observed that the policy shifts from *action* 1 to *action* 2 at state 18, suggesting that beyond this point, the preventive action is more economically sensible than enduring the system's degradation. Figure 4 shows the optimal policy obtained with this method, for which the average expected cost is of 0.1460. If we compare this ϕ to the one obtained in the previous section, we save one ten-thousandth of a unit (0.0001) in the long run, which is considerably low, but an improvement nonetheless.

4.2 Value iteration

In our value iteration approach, we incrementally refined the policy by computing V_{t+1} from V_t using the formula $V_{t+1} = \min_\alpha\{r_\alpha + P_\alpha V_t\}$. This method iteratively updated the value function until convergence, based on the rewards and transition probabilities for each action in every state.

Our implementation involved calculating the rewards for waiting and replacing in each state. The rewards for waiting or replacing and transition probabilities were computed as in the previous case.

During the value iteration, the value function and policy were updated in each iteration. The new value function was determined by choosing the minimum expected cost for each state, considering both waiting and replacing. The policy was then updated based on this new value function.

Convergence was determined by a threshold, set at $\delta(V) \approx 0.1459$, which represented the sufficient difference between successive iterations of the value function for the algorithm to converge. The process terminated once the change in the value function between iterations fell below this threshold.

After 36 iterations, the algorithm converged, revealing the optimal policy. According to our results, the policy suggests waiting in states 1 to 17 and replacing in states 18 onwards, just like in policy iteration. The convergence of both algorithms to an identical policy not only aligns with expectations but also serves as a positive indication regarding the correct implementation of each algorithm. This outcome is significant for the validation of the algorithms' accuracy and reliability.

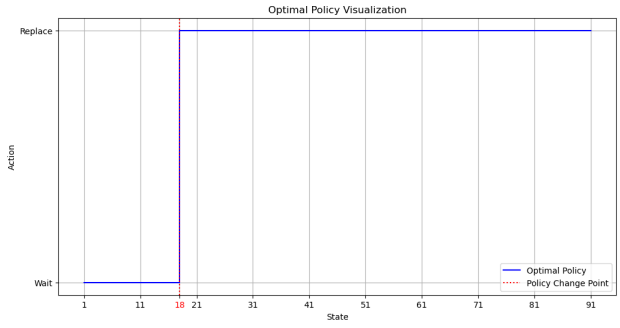

Figure 4: Optimal policy found by both methods.